

Boxu TDD

COMP710 Individual Game Project

COMP710 Individual Game Project.....	1
1. Introduction and Scope.....	1
2. Technical Algorithms and Logic	2
2.1 Movement and Collision.....	2
2.2 Level Geometry Selection.....	2
2.3 AI State Logic.....	2
2.4 Player Vision Cone/Mask.....	2
2.5 Audio and Noise Propagation.....	2
3. Data Driven Design	3
4. Diagram Explanations	3
Diagram A - Architecture.....	3
Diagram B – One frame	4
Diagram C – Startup (initialization).....	6
Diagram D – Noise Path	7
Diagram E – Collision source.....	8
Diagram F – Enemy/AI behavior state	9
5. UML Class Diagram	11
Diagram G	11
6. Debug Features	11
7. Coding Standards (and Naming Schemes)	12
8. Relevant File formats	12
9. Implemented vs Planned.....	12
10. Acceptance Test Plan.....	14
11. Risks, trade-offs, and future improvements	14
12. Bibliography	15

Diagram A - Architecture	16
Diagram B – One frame.....	17
Diagram C – Start up initialization.....	18
Diagram D – Noise Path	19
Diagram E – Collision source.....	20
Diagram F – Enemy / AI Behavior state	21
Diagram G – UML Class	23

1. Introduction and Scope

This Technical Design Document will describe the implementation and approach for the game *Boxu*. The document focuses on technical architecture, gameplay system logic, core algorithms, coding conventions, debugging, and verification strategy.

The scope includes real-time gameplay systems which have been implemented in C++ via SDL2/OpenGL, including movement, collision logic, AI state behavior, vision cone/masking, audio events (walking, hitting wall), and UI/Debug overlays.

2. Technical Algorithms and Logic

2.1 Movement and Collision

Player movement is processed per frame using delta time. Position updates are constrained against wall segments, preventing traversal through boundaries. Collision response uses the currently active wire geometry for the level, ensuring level specific physical constraints.

2.2 Level Geometry Selection

Level geometry is authored using two formats (further details in “4. Diagram Explanations, Diagram E”). The runtime systems, however, consume a single interface via ***activeWireFlat()*** and ***activeWireSegmentCount()***. This allows collision, LOS (Line of Sight), and vision cone/masking to remain consistent regardless of the authoring format.

2.3 AI State Logic

Enemy behavior is modelled as a finite state system (further details in “4. Diagram Explanations, Diagram F”). Transitions are event driven (vision/hearing) and timer driven (search/chase persistence). This improves maintainability and enables a believable pursuit and fallback behavior.

2.4 Player Vision Cone/Mask

Visibility is implemented as a full screen shader pass. Fragments outside the players' vision (or blocked by a wall) are darkened with alpha blending. This combines angular cone testing with wall occlusion checks to produce stealth style visibility feedback.

2.5 Audio and Noise Propagation

Footsteps and wall impacts trigger shared noise events through **`emitNoiseEvent()`**. Noise is used for both gameplay (enemy responding to player walking/running/hitting walls) and for debug visualization (expanding rings around the player). Footsteps are selected from a shuffle bag to reduce repetitive audio patterns, with different timing for walking vs running.

3. Data Driven Design

Data driven design is present in this project, by separating important runtime content from the core gameplay logic where practical. Rather than hardcoding every runtime asset directly into rendering code, the game loads textures, sounds, and shaders from external folders copied next to the executable. This allows assets such as splash images on start up, footstep audio clips, texture files, and shader programs to be changed without any re-writing being done to the gameplay or rendering loop.

The level geometry also used a data driven approach. Level 1 wall data is authored in `level_layout.h` using structured wall specifications, including helper functions as `Hor()` and `Ver()` for horizontal and vertical wall segments. These structured definitions are flattened into runtime wall segment data. Level 2 was made a different way to evaluate alternative level authoring and workflows, using a direct flat wall segment array, `kFacilityWireFlat`, stored in `game.cpp`.

Whilst both levels are authored differently, both are accessed at the runtime through `activeWireFlat()` and `activeWireSegmentCount()`.

This shared runtime interface is important because several systems depend on the same wall data (player collision, enemy's line of sight, wall occlusion, and the vision cone mask). By routing both level formats through the same access functions, the game keeps these systems consistent even though the source data is stored differently.

A limitation of the current implementation is that level data is still compiled into C++ source files. For future development, I believe it is worth looking into moving the level geometry and object placement into an external format such as `.json`, `.csv`, or `.txt`. This would cut down on recompiling C++ code, every time an edit is needed to be made on a level, or even a level being added.

4. Diagram Explanations

Diagram A - Architecture

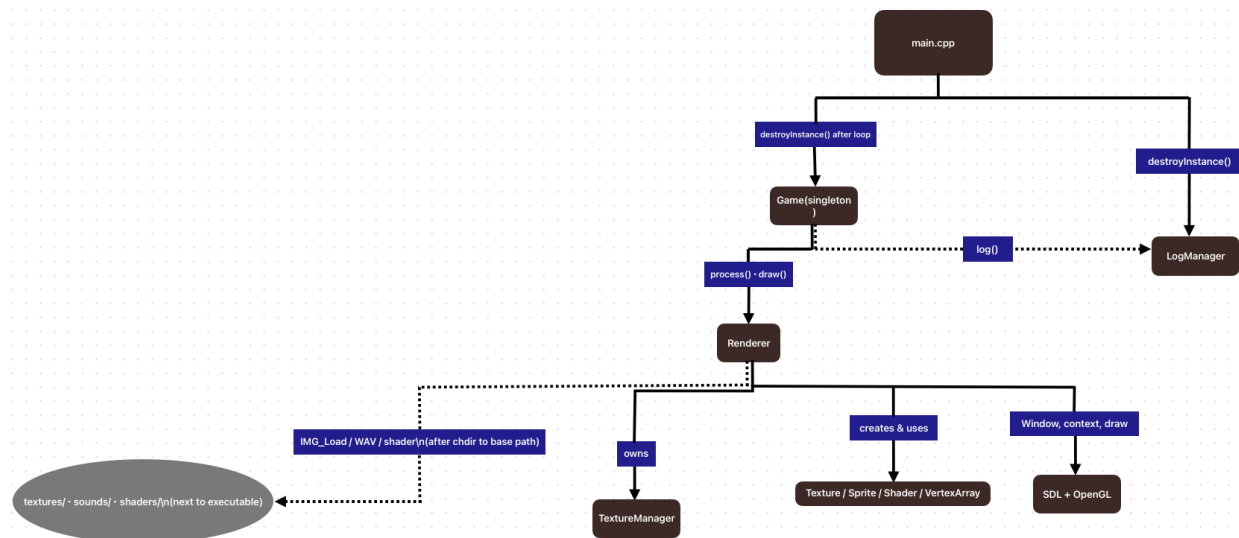


Diagram A represents the static structure of the program, not the order of execution.

- **Main.cpp** is the entry point, it starts the **Game**, runs the loop, then tears **Game** and **LogManager** down.
- **Game** is the central hub, it owns the **Renderer** pointer and calls *process* and *draw* every frame.
- **Renderer** sits on top of SDL + OpenGL: window, context, clearing, presenting, and all drawing helpers.
- **Renderer** owns a **TextureManager**, which caches **Texture** objects; **Sprite**, **Shader**, **VertexArray** are pieces used to actually draw quads and effects.
- **LogManager** is separate: **Game** (along with other codes) can log without owning the logger for the whole lifetime the same way
- The grey oval holding **Textures/ - sounds/ - shaders/** node is data on disk next to the executable. The dotted arrows mean the renderer (and game init) read those paths after the working directory is set to the app base path, NOT that those folders are CPP classes. Arrows show who owns whom, and who calls whom.

Diagram B – One frame

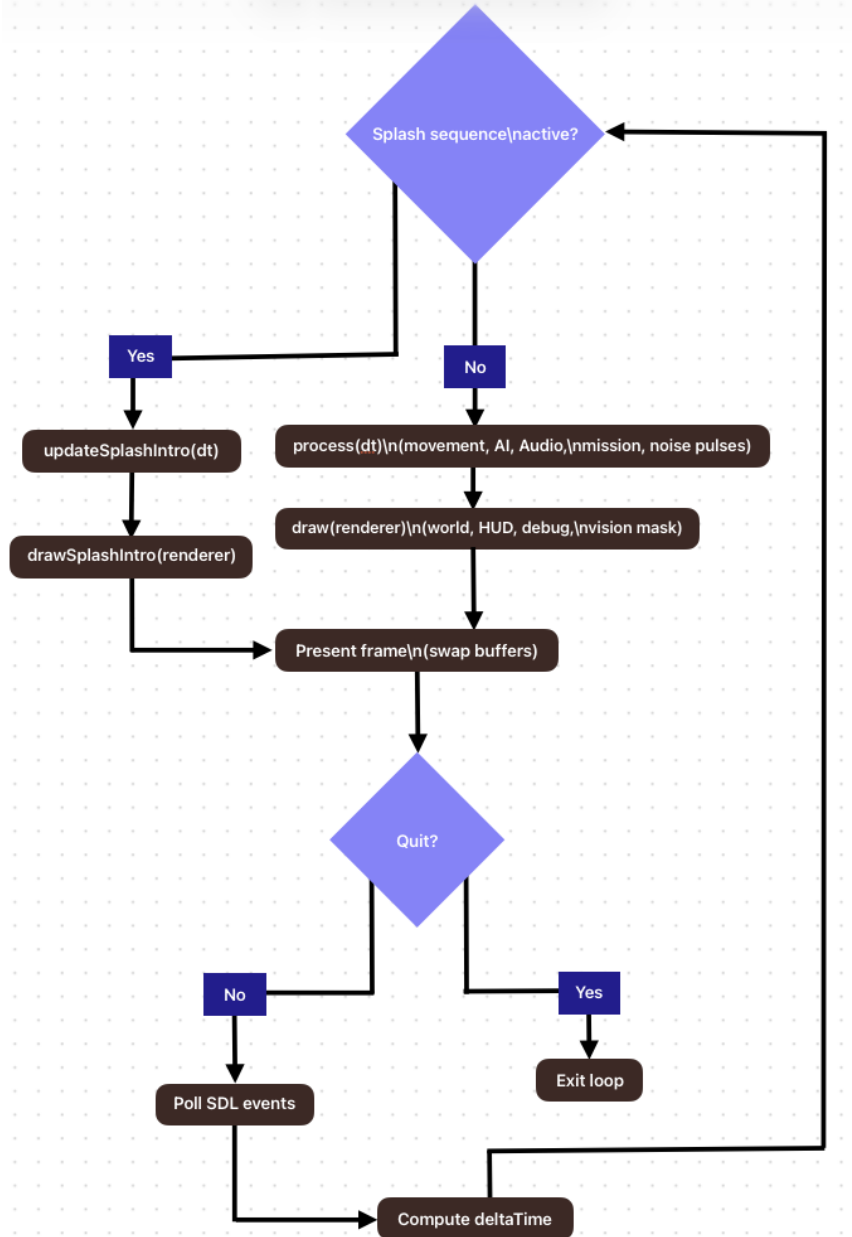


Diagram B presents what happens every frame, in order.

- First it polls the SDL events (mouse, keyboard, quit)
- Then compute **deltaTime** so movement and AI are frame rate independent
- The lavender diamond is the important branch, making sure is the splash sequence still running?
 - o Yes: It only runs **updateSplashIntro** and **drawSplashIntro** no normal gameplay, then present the frame.

- No: It runs full **process** (movement, collisions, AI, audio, mission, noise pulses), then **draw** (world, HUD, debug, vision mask), then **present**.
- This ensures the game is not running during the splash screen to start up.
- After presenting, another diamond: Quit? If not, loop back to polling events; if yes, exit the loop.

Diagram C – Startup (initialization)

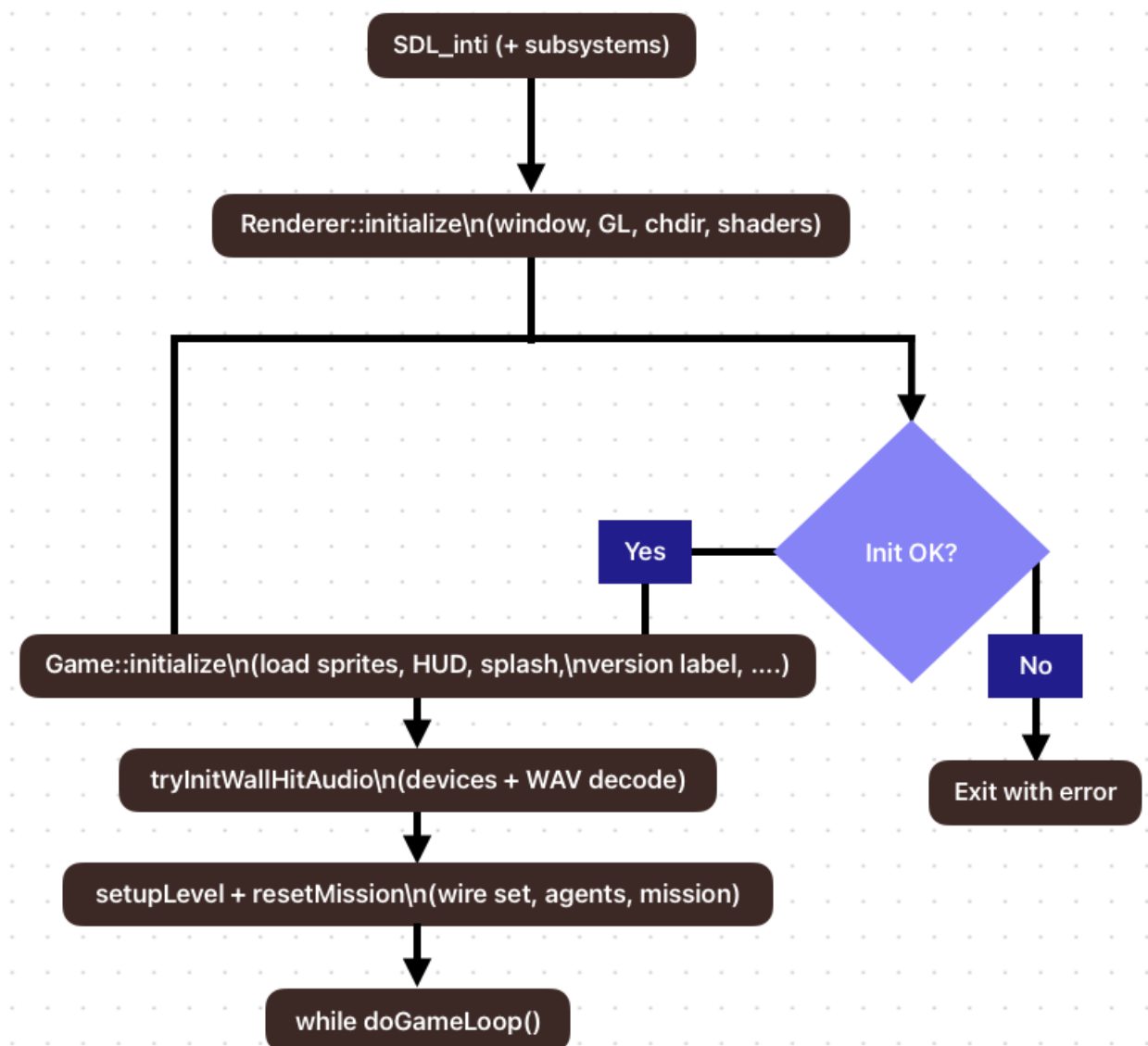


Diagram C presents the boot order, top to bottom.

- Initialize SDL (audio, video, etc..)

- Create the **window and OpenGL context and related setup in *Renderer::initialize*** (including changing to the executable directory so assets resolve).
- ***Game::initialize*** loads sprites, HUD, splash screen, version label, etc.
- Audio devices and WAV data are set up so wall hits and footsteps can play.
- ***setupLevel / resetMission*** place the player, enemies, key card, extraction zone, and pick the correct wall list for the level.
- Then it enters ***while (doGameLoop())***.

The optional “init OK?” diamond is only if it wants to show failure, such as the renderer init fails, you do not continue into game load, the program exits with an error message.

Diagram D – Noise Path

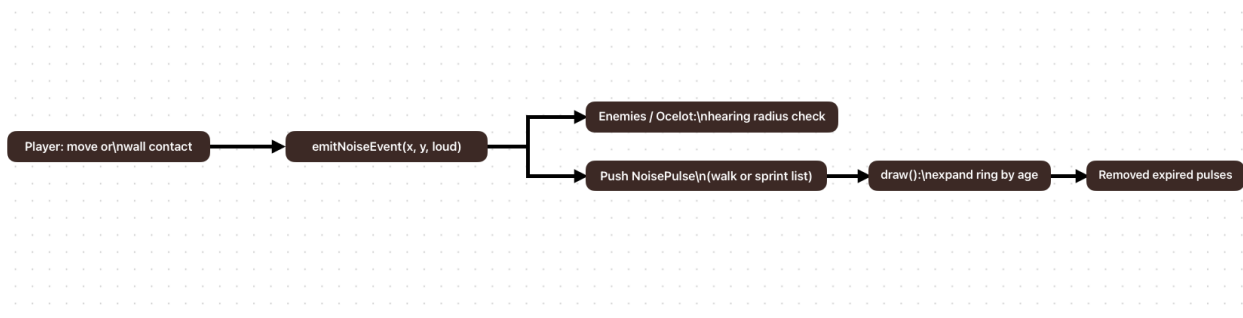


Diagram D presents the noise, which is one feature traced end to end.

- The action of the player such as footsteps when moving, or hitting a wall triggers ***emitNoiseEvent*** with a world position and whether the event is “loud”.
- From there the logic splits into two arrows:
 - o AI: enemies/Ocelot run hearing checks (distance / radius) and may change behavior if triggered.
 - o Debug visuals: a ***NoisePulse*** is pushed onto the walk or sprint list.
- On ***draw***, each pulse is drawn as an expanding ring; when its age exceeds the duration, it is removed.

Diagram E – Collision source

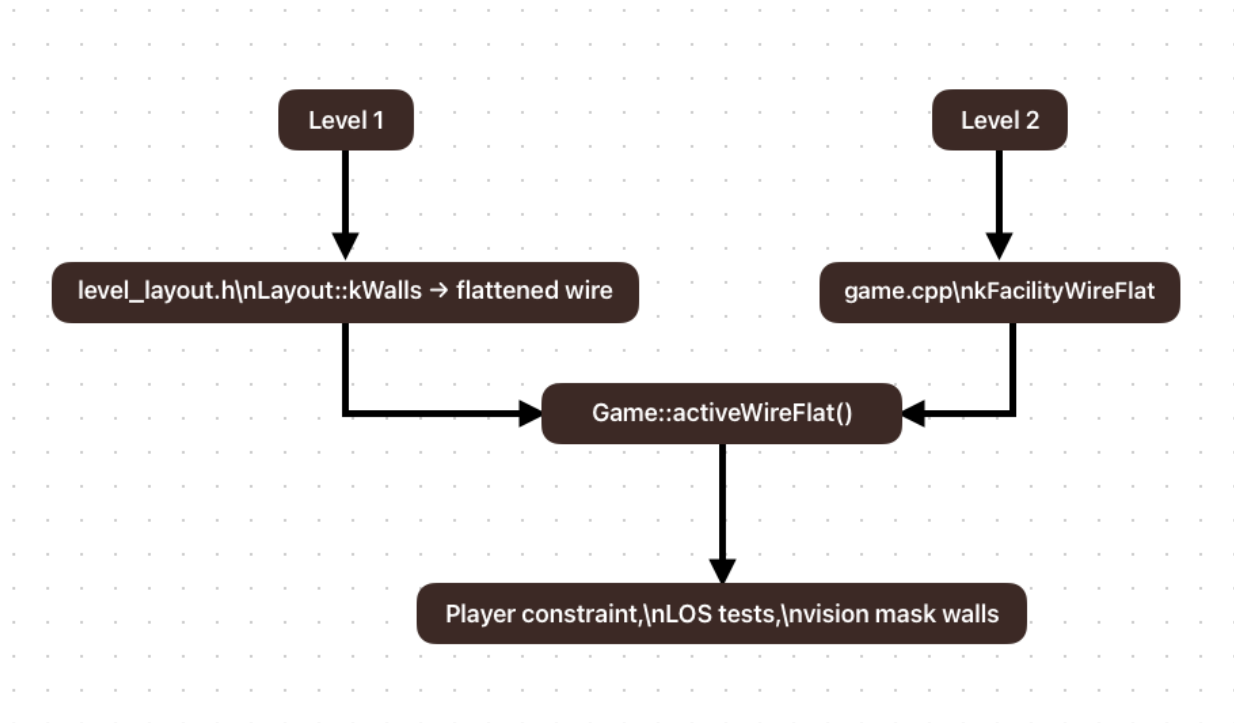


Diagram E presents the level collision, explaining how the walls work in both levels.

- Level 1 uses walls authored in *level_layout.h* as *Layout::kWalls*, then flattened at compile time into segment format used at runtime. This approach was chosen to improve readability during iterative level editing.
- Level 2 uses a separate flat array *kFacilityWireFlat* defined in *game.cpp*. In this level creation, geometry is stored directly in the runtime segment format, which made it practical for that levels layout workflow, as the level is slightly bigger.
- *Game::activeWireFlat()* selects the currently active wall list by level, and *activeWireSegmentCount()* provides the matching segment count. Collision, LOS (line of sight) checks, as well as the players vision cone system all consume this same active source, which keeps the gameplay behavior consistent across subsystems.

The use of two different authoring methods was an intentional choice. It allowed each level to be built with some experimentation of level designing on my end, in a format which was considered most suitable for the layout complexity, while still enforcing a single runtime contract for all wall dependent systems.

It is, however, worth noting that this is a poor choice in a realistic scenario. Mixed authoring styles make it inconsistent when developing more levels. It also means more hardcoded arrays and branching logic in code, which makes it more difficult to maintain. Most importantly, it would lead to more time wasted, as making a system efficient for one authoring style does not necessarily mean it has any benefits for the other.

If I were to re-do future levels, I would pick Level 1's style compared to Level 2. Due to being more legible, such as mentioning which array is horizontal and vertical, it is clear what each array does, and therefore easier to edit safely. It is also easier to extend this style with helpers, while still giving the same runtime output after flattening, so there is no performance loss for the scale of this project.

Diagram F – Enemy/AI behavior state

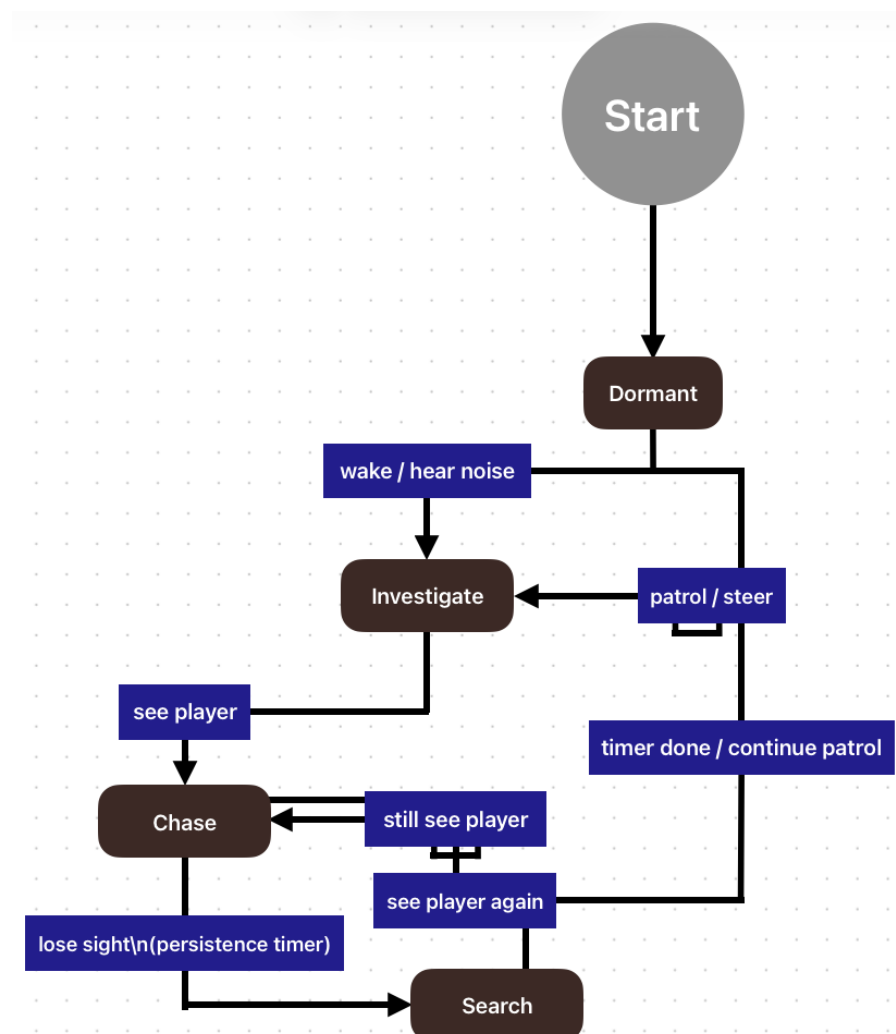


Diagram F presents the enemy/Ocelot behavior using a finite state machine with 4 stages.

Each round box / state is a mode the enemy/ocelot can be in. The arrows are transitions, and the blue labels are the reason for transitioning to a different state.

Each state represents a distinct behavior mode, and transitions are triggered by perception events (vision/hearing) or timers.

1. Dormant
 - Inactive/default waiting mode (if used for that enemy type).
2. Investigate
 - The enemy moves toward patrol or last known/noise location.
3. Chase
 - Entered when the player is within the enemy LOS (line of sight). This will tune the movement speed and the direction facing the player in a pursuing manner.
 - Chase looping itself, would mean the enemy will remain in **Chase** state while the player remains within their LOS.
4. Search
 - **Chase** to **Search** with “Lose sight + Persistence” makes the experience more realistic as the enemy does not instantly forget the player when they hide/move out of the enemy’s LOS.

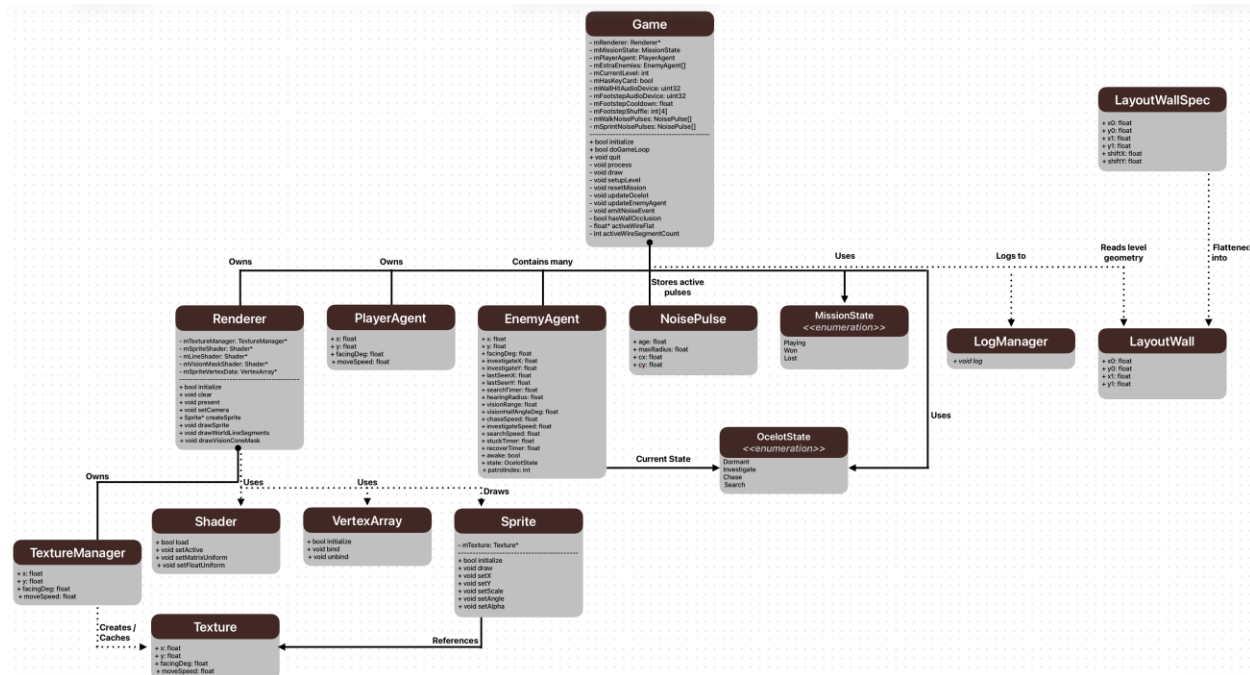
The most important transition is **Chase** to **Search** after vision is lost, this is controlled by a cooldown timer before they stop chasing.

This prevents agents from giving up immediately and creates a more believable pursuit behavior from the enemy.

From **Search**, the agent returns to **Investigate** when the timer expires, or re-enters **Chase** if the player is seen again. This state based approach was chosen to keep AI logic modular and explainable, whilst still supporting dynamic gameplay reactions.

5. UML Class Diagram

Diagram G



The **Game** class acts as the main orchestration layer, owning the Renderer, player state, enemy list, mission state, audio handles, and active noise pulse collections. Renderer manages the rendering pipeline and owns the TextureManager, while relying on Sprite, Texture, Shader, and VertexArray for drawing. Gameplay specific structures such as PlayerAgent, EnemyAgent, MissionState, OcelotState, and NoisePulse are included to show how AI, mission flow, and noise/debug feedback are represented in code. The diagram also includes wall geometry structures to show how level data is connected to collision, line of sight, and vision cone/masking systems.

6. Debug Features

Implemented debug support includes:

- Runtime debug view toggle.
- Hitbox and helper overlays for spatial verification.
- Noise pulse visualizers for footsteps and wall impacts (green for small noise, blue for loud noises).

- Vision cone visualization behavior through mask logic.
- Pulsing HUD indicator to confirm debug mode state.
- Logging via **LogManager** for startup/load/runtime diagnostics.

These implemented features support faster verification of collision, AI reaction radius, and general visual system correctness.

7. Coding Standards (and Naming Schemes)

This project follows a consistent C++ style with:

- Clear separation of declarations (**.h**) and implementations (**.cpp**).
- Descriptive identifiers for game systems (e.g **mOcelotState**, **mFootstepCooldown**).
- Constant naming with **k** prefix for immutable values (e.g **kChasePersistenceSec**).
- Member naming with **m** prefix for instance fields.
- Enum classes for state safety (**MissionState**, **OcelotState**).
- Defensive failure path cleanup for memory/resource safety.

With these structures and naming schemes, the aim is to improve readability and maintainability under iterative development.

8. Relevant File formats

Key formats that were used in the development/runtime include:

- C++ source/header: **.cpp**, **.h**
- Build config: **CMakeLists.txt**
- Shaders: **.vert**, **.frag**
- Textures: **.png**, **.jpg**, **.jpeg**
- Audio: **.wav**
- Documentation: **.pdf**, **.txt**
- Version control log: Git commit history, exported as **.txt**

9. Implemented vs Planned

Feature	Status	Notes
Splash screen	Implemented	AUT splash, followed by the games splash screen are loaded from texture assets and displayed before gameplay.
Core movement and collision	Implemented	Player movement is constrained by active level wall geometry; therefore the player cannot just walk through walls, nor can enemies.
Mission win / lose loop	Implemented	The player must collect the key card and reach the extraction point. This progresses to the next level. Winning (extracting) and losing (being caught) have been implemented, along with pressing R to restart.
Vision cone / mask	Implemented	A fullscreen shader darkens areas outside the players' visible cone and accounts for wall occlusion.
Enemy AI state system	Implemented	Enemies use investigate, chase, and search behavior with vision / hearing triggers.
Chase persistence	Implemented	Enemies do not immediately stop chasing when the player is out of their line of sight.
Footstep audio system	Implemented	Four footstep clips are played in shuffled order, with different timing for walking and sprinting.
Noise / Hearing system	Implemented	Footsteps and wall impacts create noise events that can alert enemies.
Debug visualizers	Implemented	Debug mode shows hitboxes, noise pulses, and a pulsing "DEBUG VIEW" indicator.
Memory / resource cleanup	Implemented	Failure paths and shutdown logic clean up render, textures, sprites, and audio buffers.
External level loading	Planned (Future experiment)	Current level geometry is still compiled into C++ source; a future version would load level data from external files.
Unified level authoring format	Planned (Future experiment)	Level 1 and Level 2 uses different authoring styles. These will be standardized in future projects, or continued development of this game.
A third level	Planned (Future experiment)	The current architecture was designed with more levels in mind, however it is planned to be added via external level loading method. This could be considered another experiment yet to be tried.

More advanced pathfinding	Planned (Future experiment)	Current AI uses steering candidates and wall occlusion. Future work could potentially use waypoint graphs.
---------------------------	-----------------------------	--

10. Acceptance Test Plan

#	Test Question	Pass Condition
1	When the executable is launched, does the game display the AUT splash screen followed by the Boxu splash screen before gameplay begins?	Pass if both splash screens appear in order, fade correctly, and gameplay does not start until the splash sequence finishes.
2	When pressing W/A/S/D, does the player move in the correct direction, and does mouse movement rotate the player smoothly toward the cursor?	Pass if movement responds immediately, respects delta time, and facing direction follows mouse input without snapping incorrectly.
3	When the player moves into walls or boundaries in either level, is the player prevented from passing through the level geometry?	Pass if the player remains inside the playable space and collision works consistently in both Level 1 and Level 2.
4	Can the player complete the mission loop by collecting the key card and reaching the extraction zone?	Pass if the key card can be collected, extraction is detected, Level 1 progresses to Level 2, and Level 2 can end in a win state.
5	Can the player lose the game when caught or detected under the intended fail conditions, and restart without closing the executable?	Pass if the lose state appears correctly and pressing the restart input resets the mission/game state without relaunching.
6	When the player enters an enemy's line-of-sight, does the enemy transition into chase behavior and pursue the player?	Pass if the enemy changes from investigate/search into chase, faces/moves toward the player, and can trigger a loss condition.
7	When the player breaks line-of-sight during a chase, does the enemy continue briefly before searching instead of stopping instantly?	Pass if chase persistence is noticeable, then the enemy enters search/investigate behavior after the timer expires.
8	When the player walks, sprints, or hits a wall, do audio cues play and do enemies react to nearby noise events?	Pass if footsteps/wall sounds are audible, sprint cadence is faster, and nearby enemies investigate/chase based on noise range.
9	When debug mode is enabled, are hitboxes, noise pulses, and the pulsing "DEBUG VIEW" label displayed correctly?	Pass if debug overlays appear only in debug mode and noise pulses expand/expire visually without canceling each other early.
10	During a normal play session through both levels, does the release build remain stable without crashes, obvious runtime bugs, or memory leak symptoms?	Pass if the game can be played through normal win/loss/restart flows without crashing, hanging, or steadily degrading performance.

11. Risks, trade-offs, and future improvements

A trade off done early on was using two different level authoring formats. While this was partly experimental on my part to learn from this, it did increase maintenance cost (time

wise). For future levels, geometry should be unified under one data-driven schema to reduce duplication and branching logic.

12. Appendix

CMake. (n.d.). *CMake documentation*. <https://cmake.org/documentation/>

Cppreference. (n.d.). *C++ reference*. <https://en.cppreference.com/>

Khronos Group. (n.d.). *OpenGL reference pages*. <https://registry.khronos.org/OpenGL-Refpages/>

Mermaid. (n.d.). *Mermaid documentation*. <https://mermaid.js.org/intro/>

Simple DirectMedia Layer. (n.d.). *SDL2 documentation*.
<https://wiki.libsdl.org/SDL2/FrontPage>

Simple DirectMedia Layer. (n.d.). *SDL_image 2.0 documentation*.
https://wiki.libsdl.org/SDL2_image/FrontPage

The OpenGL Extension Wrangler Library. (n.d.). *GLEW: The OpenGL Extension Wrangler Library*. <https://glew.sourceforge.net/>

Full size versions of all diagrams are included below for readability.

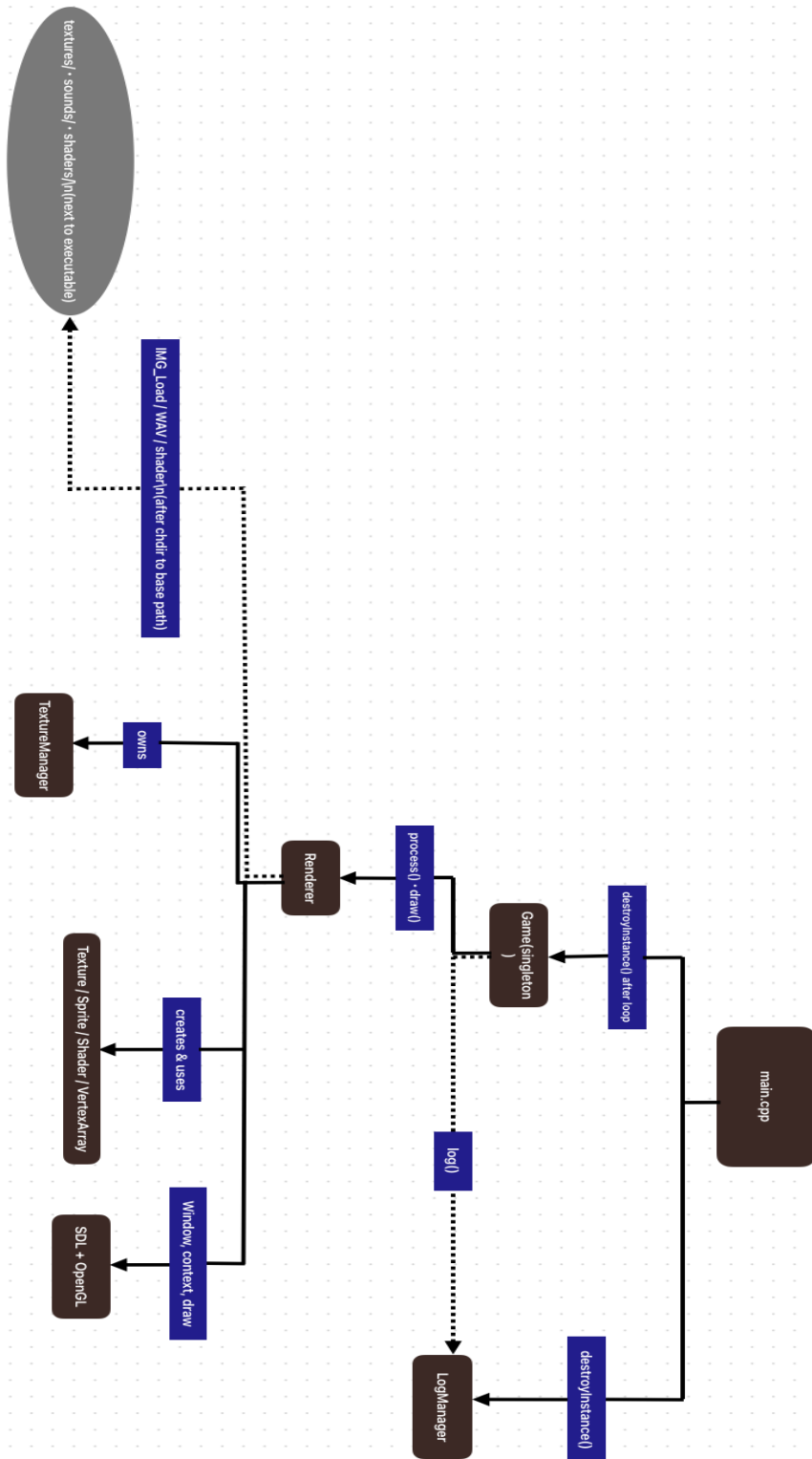


Diagram A - Architecture

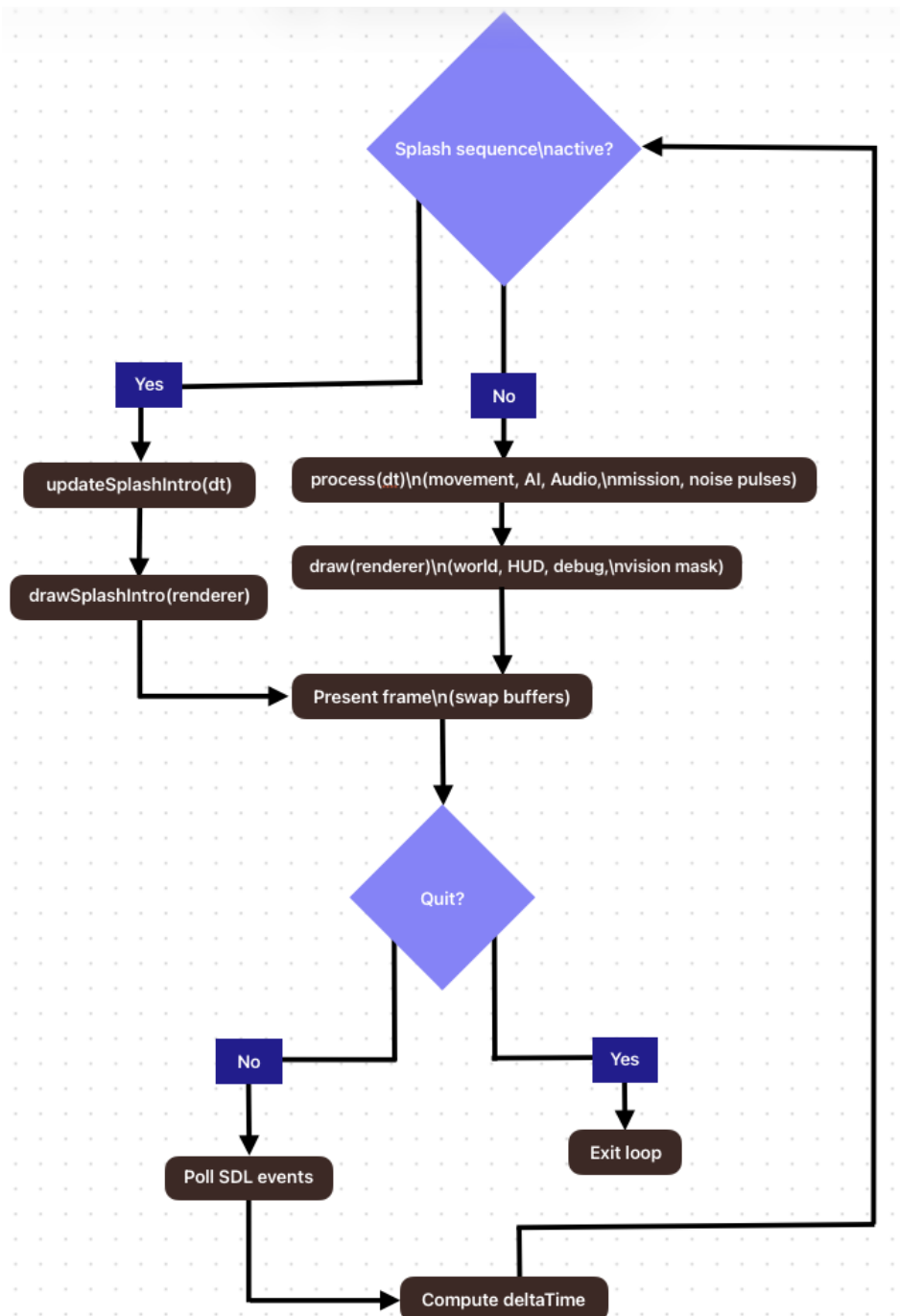


Diagram B – One frame

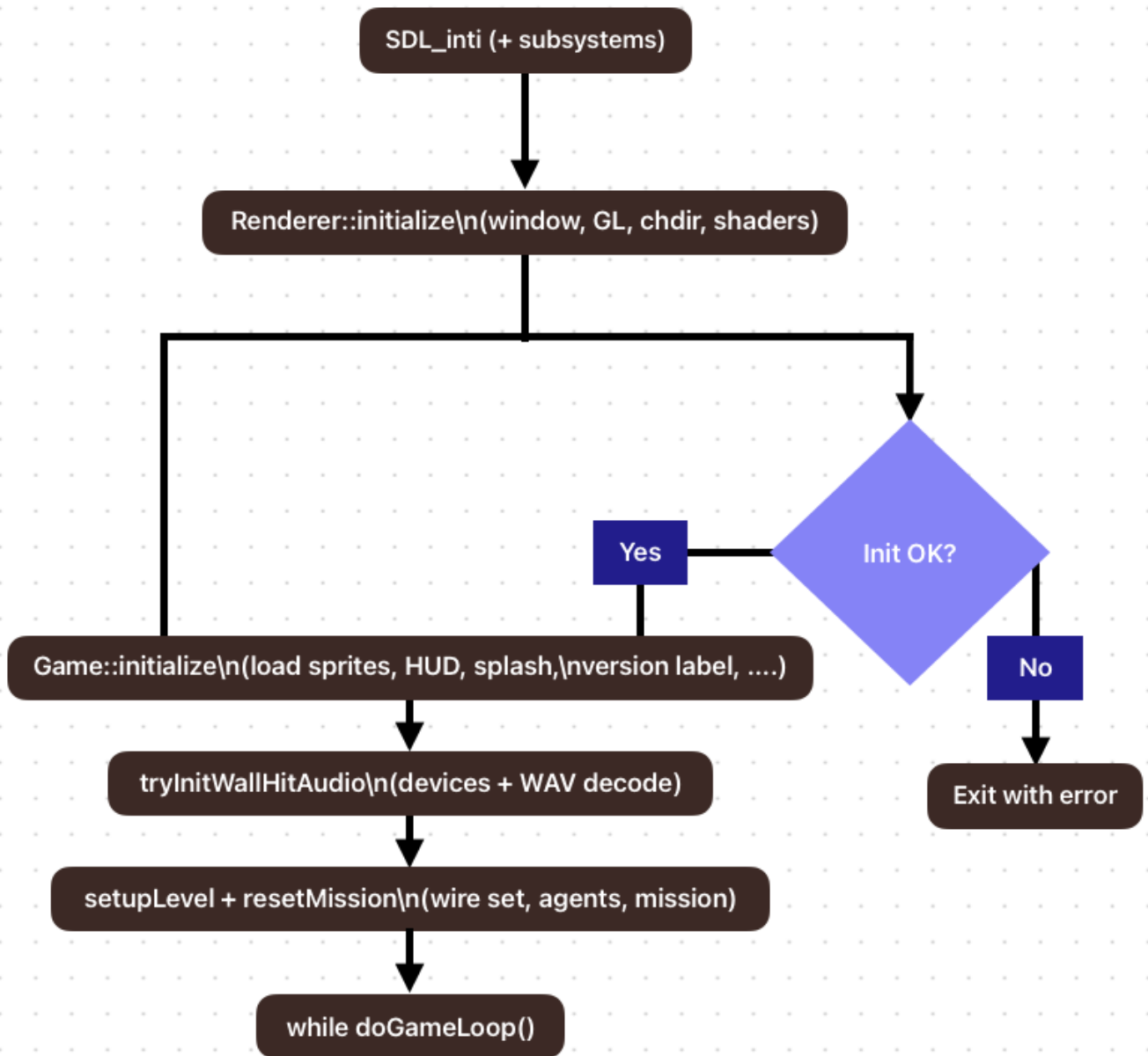


Diagram C – Start up initialization

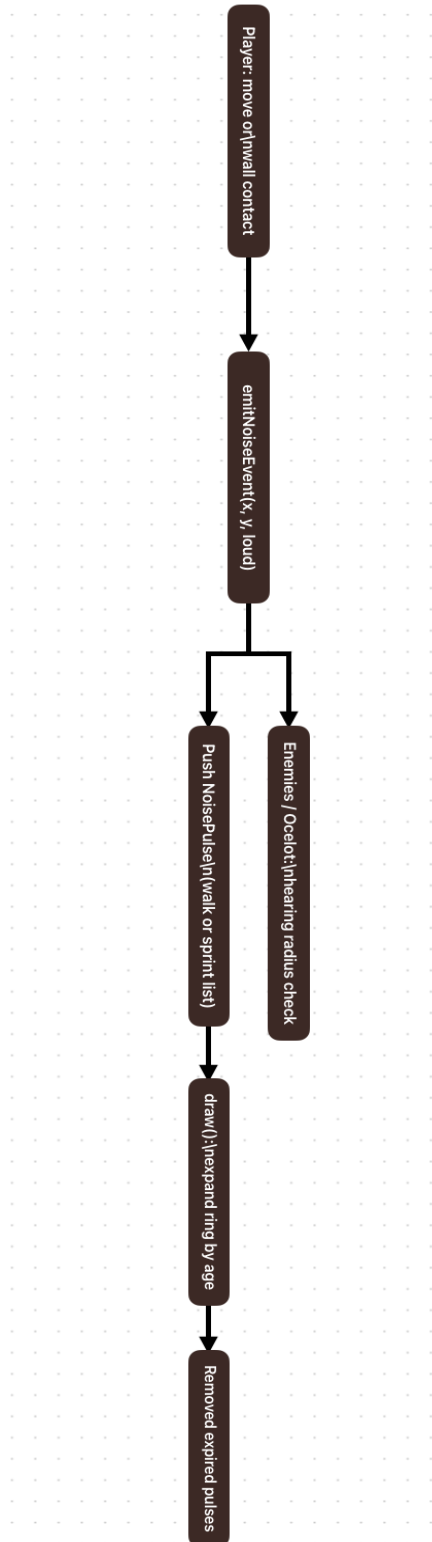


Diagram D – Noise Path

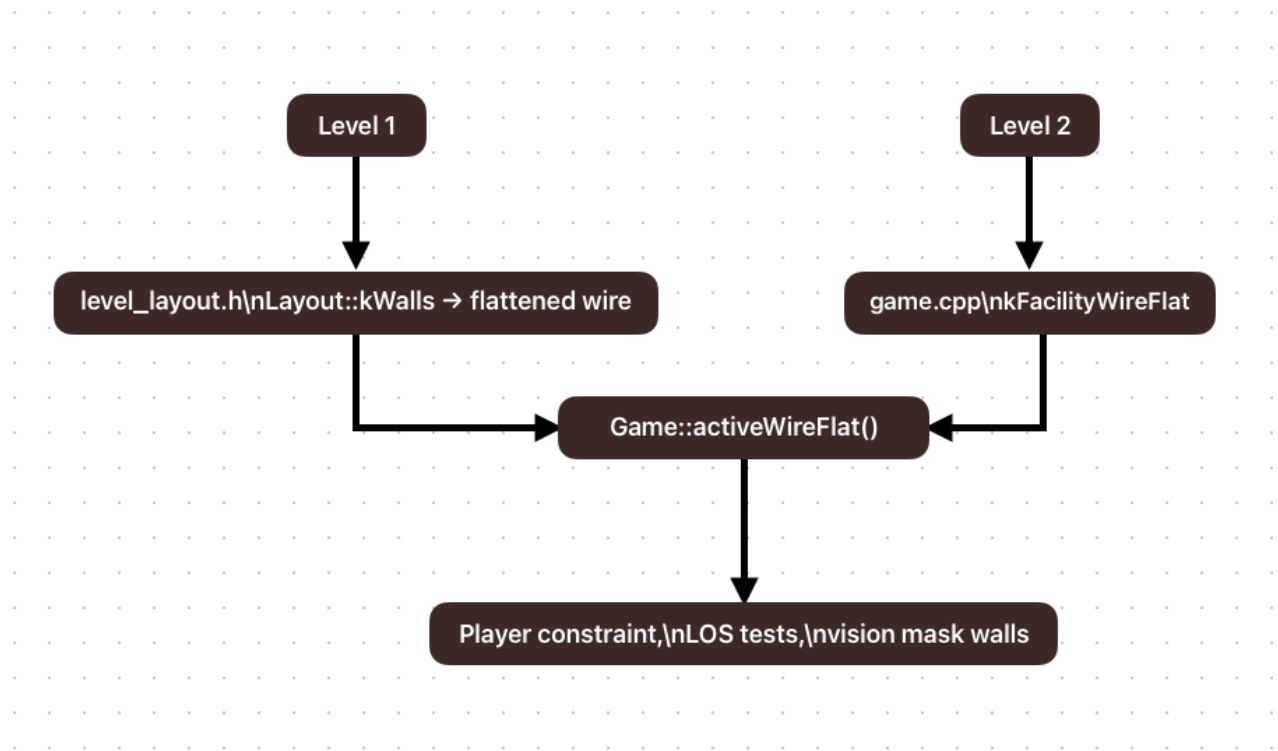


Diagram E – Collision source

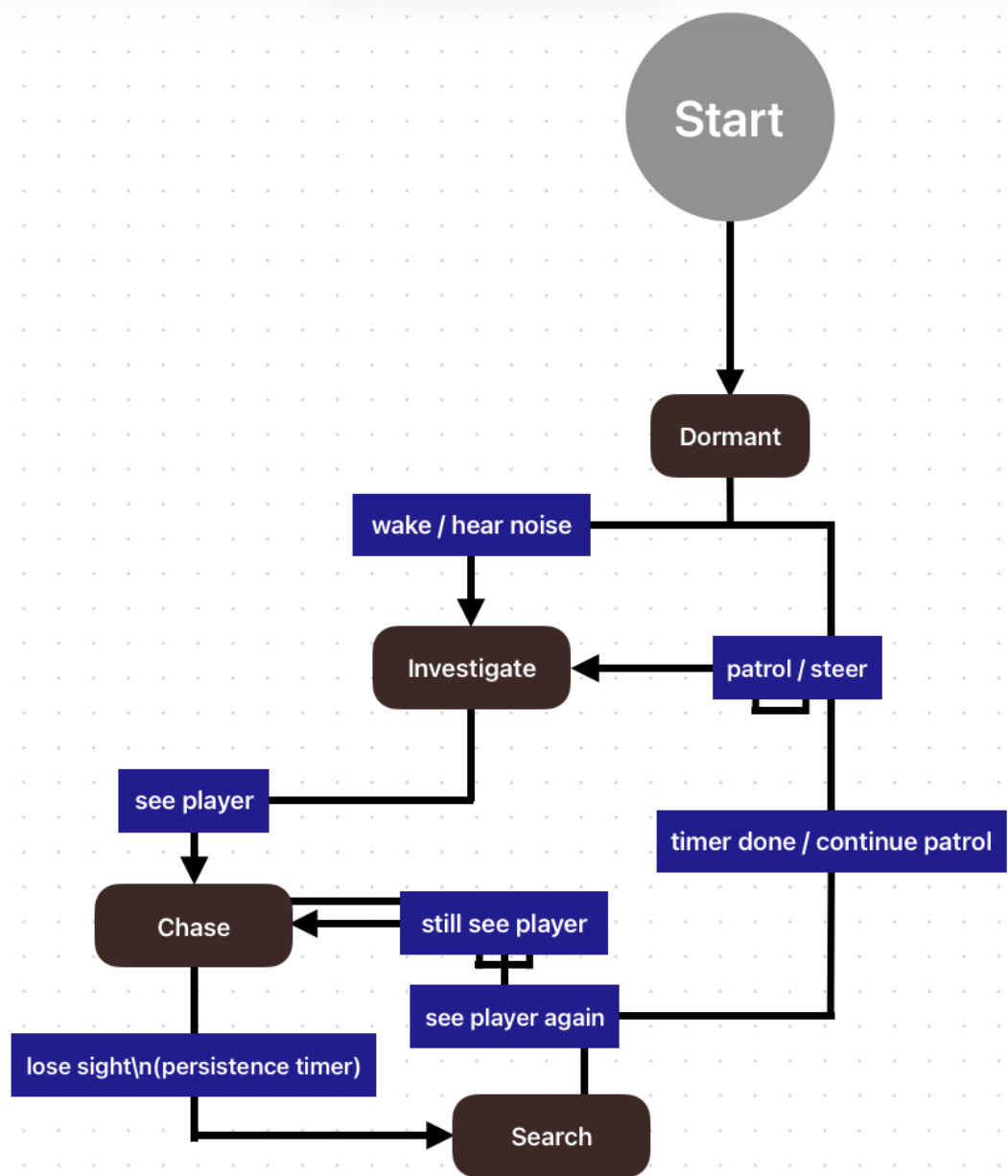
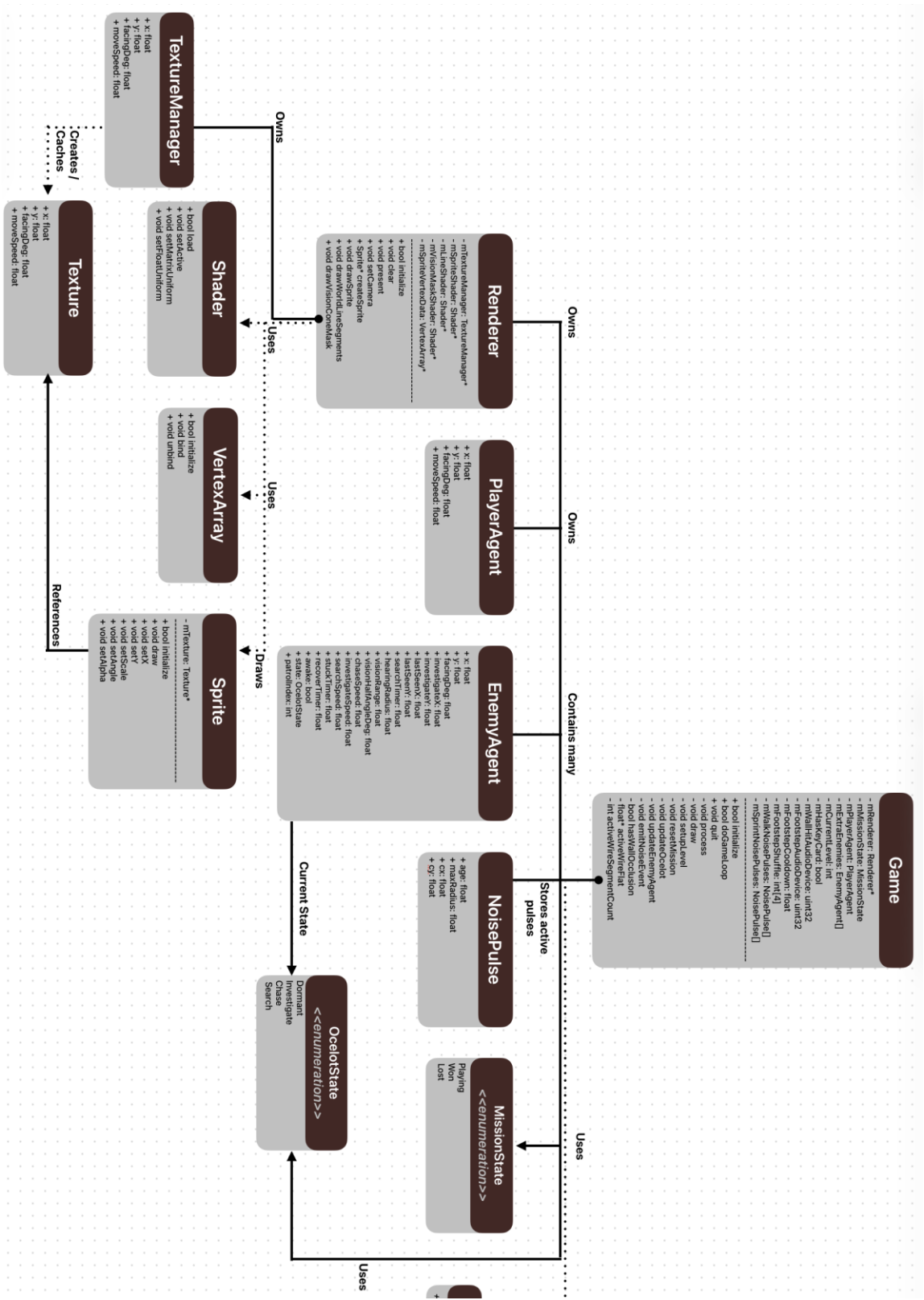


Diagram F – Enemy / AI Behavior state



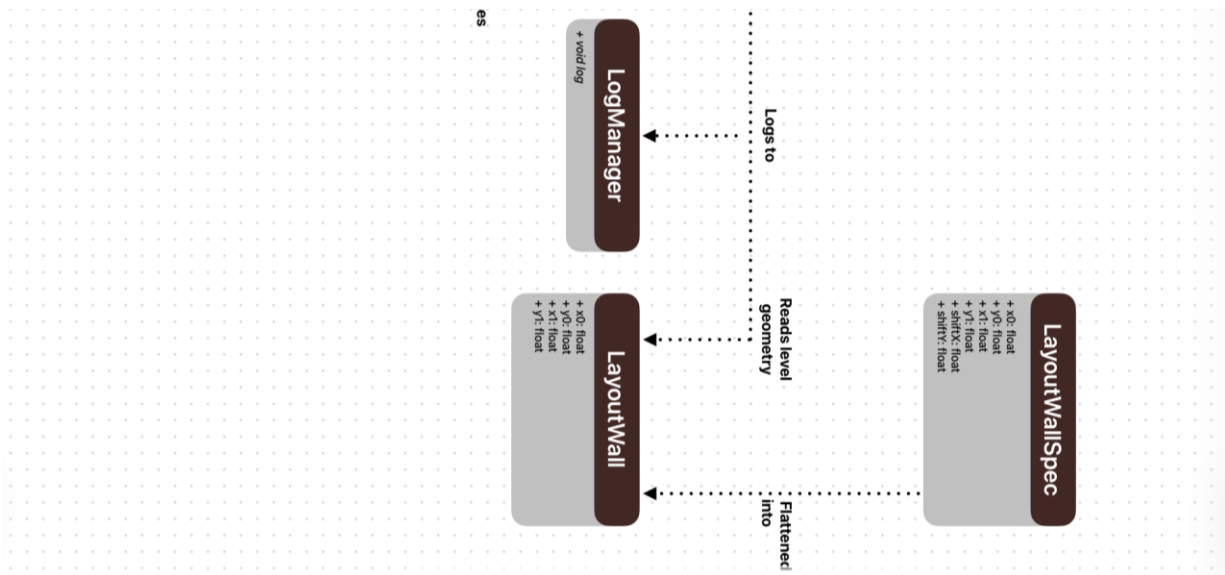


Diagram G – UML Class